SAVEETHA SCHOOL OF ENGINEERING
C PROGRAMMING ASSIGNMENT – Logic Analysis and Design Challenges

Field	Entry
Student Name	BAVADARANI S
Register Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Logic Analysis and Design Challenges (Q1–Q7, 50 marks)
GitHub Repository URL	https://github.com/bavadarini28 /CSA0209-1912260562-BAVADARANI.S.git
Submission Date	12 NOVEMBER 2026

Q1 – Precedence, Associativity and the Discount Bug

Predictions before running (by the rules)

Precedence (high → low): * / (same level, left-to-right), then + − (left-to-right), then > , then ?: (right-to-left), then = (right-to-left). The rule I apply to total = price*qty - price*qty*15/100*senior + price*qty*5/100 is:

```
total = ((price*qty) - ((((price*qty)*15)/100)*senior)) + (((price*qty)*5)/100)
```

Order: (1) price*qty; (2) ×15; (3) integer-divide by 100 (the fraction is truncated); (4) ×senior; the subtraction is then done; (5) the right-hand term price*qty*5/100 is built the same way; (6) the final + is evaluated left to right; (7) > compares total with 1000; (8) ?: picks total−50 or total; (9) = stores the result.

No.	price, qty, senior	Predicted total	Predicted net	Observed total / net	Pharmacist intended (net)
1	250, 4, 1	900	900	900 / 900	Rs 892.50
2	120, 5, 0	630	630	630 / 630	Rs 630.00
3	999, 1, 1	899 (999 − 149 + 49)	899	899 / 899	Rs 891.61
4	500, 3, 1	1350	1300	(by hand) 1350 / 1300	Rs 1288.75

Bill 3 by hand: 999*15 = 14985; /100 = 149 (true 149.85); ×1 = 149. GST term 999*5 = 4995; /100 = 49 (true 49.95). Total = 999 − 149 + 49 = 899. Bill 4: discount 1500*15/100 = 225, GST term 1500*5/100 = 75, total = 1350 > 1000, so net = 1300.

Expressions that differ from what the pharmacist intended

- **GST is charged on the undiscounted amount.** The code adds 5% of price*qty; the intended rule (assumption: GST applies to the discounted value) is 5% of price*qty − discount. Bill 1: 900 vs 892.50.
- **Integer division truncates paise.** 15/100 style divisions drop the fraction (149.85 → 149, 49.95 → 49). Writing price*qty*(15/100) would be even worse: 15/100 = 0, so the discount would vanish.
- **The Rs 50 rebate uses the wrong total,** so bills near Rs 1000 can fall on the wrong side of the threshold (e.g. buggy total 1000, correct total 1050).
- **Money is held in int rupees;** no paise can be shown.

Inquiry – count = count++ + 1

The statement both modifies count (the ++) and reads count (the + 1 operand) with no sequence point between them, so the two accesses are unsequenced. The C standard says the behaviour is undefined: the program may give 6, 7 or anything. On my gcc 13 it printed 6, but another compiler or optimisation level may differ. With -Wall gcc reports:

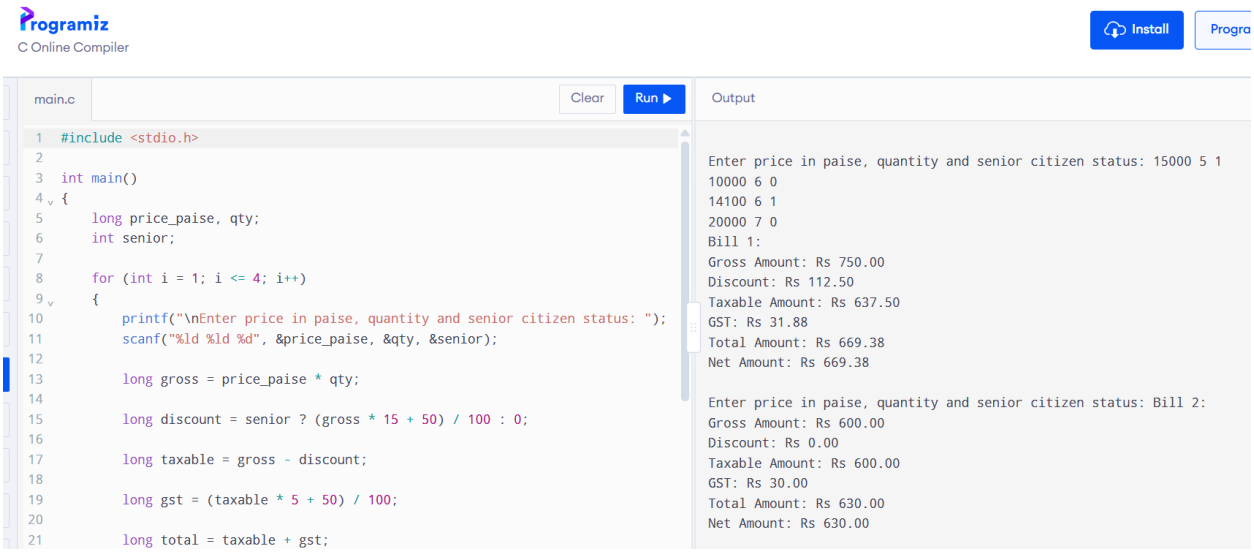
```
q1.c:13:21: warning: operation on 'count' may be undefined [-Wsequence-point]
```

Fix: write count = count + 1; or count++; (never both).

Design decision – corrected calculation (paise, brackets and intermediate variables)

```
long gross   = (long)price_paise * qty;
long discount = senior ? (gross * 15 + 50) / 100 : 0;   /* round half-up */
long taxable  = gross - discount;
long gst      = (taxable * 5 + 50) / 100;
long total    = taxable + gst;
long net      = (total > 100000L) ? (total - 5000L) : total;  /* Rs 1000, Rs 50 */
```

Output for the four bills: Rs 892.50, Rs 630.00, Rs 891.61 and net Rs 1288.75 – matching the hand-calculated intended values. I use both: intermediate variables so that each business rule (gross, discount, GST, rebate) is a visible, testable line, and brackets only where precedence is not obvious to a reader (e.g. (gross * 15 + 50) / 100). Paise in long removes the truncation error.



Alternative Concept and Complexity Analysis

Several statements with temporaries gross, discount, tax. Time: the same arithmetic operations (about 3 multiplications, 3 divisions, 3 additions) – compilers typically produce identical machine code, so O(1) either way. Space: 3–5 extra variables, i.e. 24–40 bytes of stack, usually in registers: O(1). Correctness improves because the order of evaluation is explicit and each step can be printed and checked; readability improves because the code mirrors the business rules, and a bug such as the GST base is visible at once.